

L07: Scene Graphs

Chris Carvelli

Recap

GUI

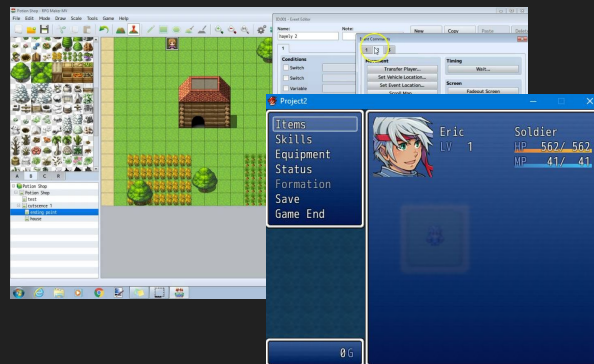
```
+ InputBox(val : &string)
+ Button(lbl : string) : bool
  [...]

// some API use return values
// instead of reference vals
+ InputBox(val : string) : string
```

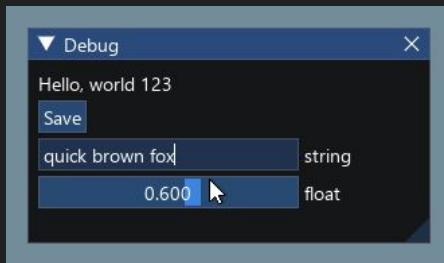
InputBox

```
- id : long
- text : string
+ onInputChanged <event>
+ setValue(val : string)
```

Immediate vs Retained
GUIs



Tools vs Game
GUIs



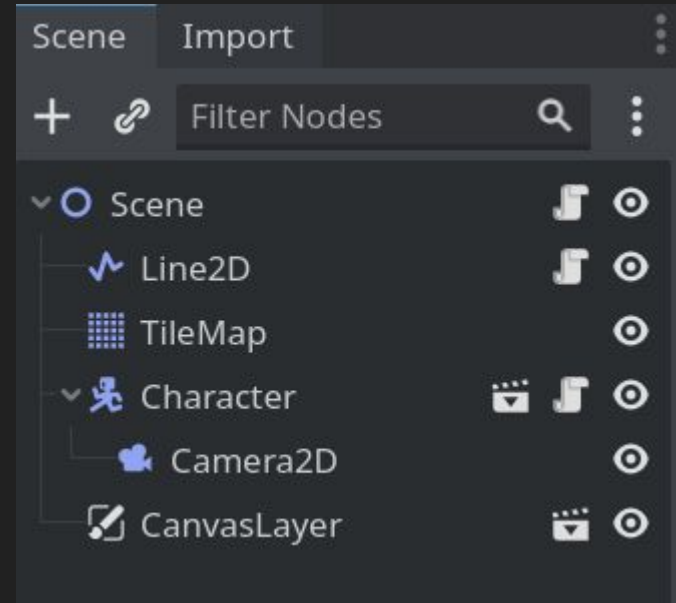
dear ImGui

Exercise 06 review

Problem statement: How do we...



- ...define game content in data rather than code?
- ...move as much responsibility from the game code into the engine code?



- ...compute complex transformations fast?
- ...express transform relationships between objects in an intuitive way?

Agenda

1. Matrices
2. Transforms revised
3. Data-driven Scenes

Agenda

1. Matrices
2. Transforms revised
3. Data-driven Scenes

Current way to change frames of reference

```
// converts the given point to the viewport of the given camera
vec2f point_global_to_screen(SDLContext* context, vec2f p)
{
    SDL_assert(context);
    Camera* camera = context->camera_active;

    SDL_assert(camera);

    vec2f camera_size;
    camera_size.x = (context->window_w / camera->pixels_per_unit) * camera->normalized_screen_size.x;
    camera_size.y = (context->window_h / camera->pixels_per_unit) * camera->normalized_screen_size.y;

    vec2f camera_offset;
    camera_offset.x = (context->window_w / camera->pixels_per_unit) * camera->normalized_screen_offset.x;
    camera_offset.y = (context->window_h / camera->pixels_per_unit) * camera->normalized_screen_offset.y;

    vec2f ret = p;
    ret = ret - camera->world_position;
    ret = ret * camera->zoom;
    ret = ret + camera_size / 2;
    ret.y = camera_size.y - ret.y;
    ret = ret * camera->pixels_per_unit + camera_offset;

    return ret;
}

SDL_FRect rect_global_to_screen(SDLContext* context, SDL_FRect rect);
vec2f point_global_to_screen(SDLContext* context, vec2f p);
vec2f point_screen_to_global(SDLContext* context, vec2f p);
vec2f point_screen_to_window(SDLContext* context, vec2f p);
vec2f point_window_to_screen(SDLContext* context, vec2f p);
// ...
```

Current way to change frames of reference

// converts the given point to the viewport of the given camera

```
vec2f point_global_to_screen(SDLContext* context, vec2f p)
```

```
{
```

```
    SDL_assert(context);
```

```
    Camera* camera = context->camera_active;
```

```
    SDL_assert(camera);
```

```
    vec2f camera_size;
```

```
    camera_size.x = (context->window_w / camera->pixels_per_unit) * camera->normalized_screen_size.x;
```

```
    camera_size.y = (context->window_h / camera->pixels_per_unit) * camera->normalized_screen_size.y;
```

```
    vec2f camera_offset;
```

```
    camera_offset.x = (context->window_w / camera->pixels_per_unit) * camera->normalized_screen_offset.x;
```

```
    camera_offset.y = (context->window_h / camera->pixels_per_unit) * camera->normalized_screen_offset.y;
```

```
    vec2f ret = p;
```

```
    ret = ret - camera->world_position;
```

```
    ret = ret * camera->zoom;
```

```
    ret = ret + camera_size / 2;
```

```
    ret.y = camera_size.y - ret.y;
```

```
    ret = ret * camera->pixels_per_unit + camera_offset;
```

```
    return ret;
```

```
}
```

FINICKY
ERROR PRONE
INEFFICIENT

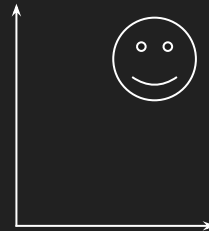
```
SDL_FRect rect_global_to_screen(SDLContext* context, SDL_FRect rect);  
vec2f point_global_to_screen(SDLContext* context, vec2f p);  
vec2f point_screen_to_global(SDLContext* context, vec2f p);  
vec2f point_screen_to_window(SDLContext* context, vec2f p);  
vec2f point_window_to_screen(SDLContext* context, vec2f p);  
// ...
```

Matrices...

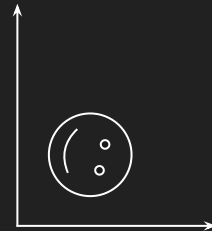
Perform operations on vectors and points using linear transformations only (addition and multiplication), which are very fast compared to other mathematical operations such as division, exponentiation/logarithms and trigonometric functions.

We will use them for our 3 operations

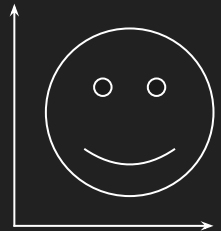
- Rotation
- Scale
- Translation



Translate



Rotate



Scale

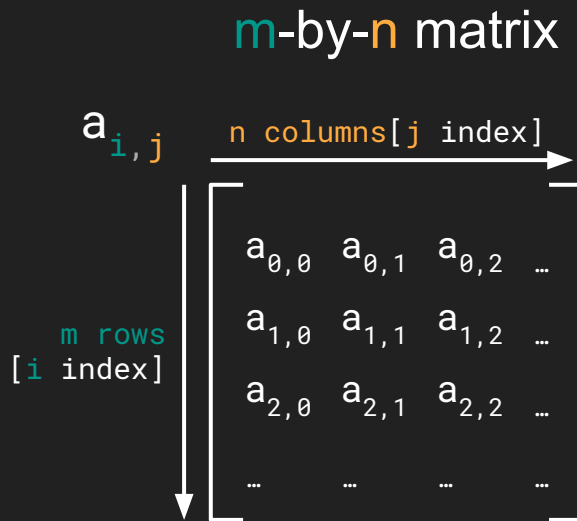
...and why we need them

Using matrices for linear transformations gives

- Clean code
- Uniform interface
- Higher performance
- Works well for 2D and 3D (as we will see next time)

Matrix representation

- 2 dimensional array of numbers
- We usually focus on small matrices (up to 4x4)



Matrix multiplication

The diagram shows the multiplication of a 2×3 matrix by a 3×2 matrix to produce a 2×2 matrix. The first matrix is $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ with dimensions 2×3 below it. The second matrix is $\begin{bmatrix} 1 & 1 \\ 10 & 2 \\ 100 & 3 \end{bmatrix}$ with dimensions 3×2 below it. The result matrix is $\begin{bmatrix} 321 & ? \\ ? & ? \end{bmatrix}$ with dimensions 2×2 below it. Orange boxes highlight the first row of the first matrix, the first column of the second matrix, and the top-left element of the result matrix. Two orange curved arrows show the dot product of the first row of the first matrix with the first column of the second matrix to calculate the top-left element of the result matrix.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 1 & 1 \\ 10 & 2 \\ 100 & 3 \end{bmatrix} = \begin{bmatrix} 321 & ? \\ ? & ? \end{bmatrix}$$

$2 \times 3 \qquad \qquad 3 \times 2 \qquad \qquad 2 \times 2$

Example:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \times \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 4 \\ 10 & 8 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix} \times \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 7 & 10 \end{bmatrix}$$

Matrix multiplication

A diagram illustrating matrix multiplication. It shows a 2×3 matrix multiplied by a 3×2 matrix to result in a 2×2 matrix. The first matrix is $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ with dimensions 2×3 below it. The second matrix is $\begin{bmatrix} 1 & 1 \\ 10 & 2 \\ 100 & 3 \end{bmatrix}$ with dimensions 3×2 below it. The result matrix is $\begin{bmatrix} 321 & ? \\ ? & ? \end{bmatrix}$ with dimensions 2×2 below it. Orange boxes highlight the first row of the first matrix, the first column of the second matrix, and the top-left element of the result matrix. Two orange curved arrows show the dot product: one from the first row of the first matrix to the first column of the second matrix, and another from the result matrix's top-left element back to the same dot product.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 1 & 1 \\ 10 & 2 \\ 100 & 3 \end{bmatrix} = \begin{bmatrix} 321 & ? \\ ? & ? \end{bmatrix}$$

$2 \times 3 \qquad \qquad 3 \times 2 \qquad \qquad 2 \times 2$

Example:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \times \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 4 \\ 10 & 8 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix} \times \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 7 & 10 \end{bmatrix}$$

Matrix multiplication

The diagram illustrates the first step of matrix multiplication. It shows a 2×3 matrix multiplied by a 3×2 matrix to produce a 2×2 result matrix. The first row of the first matrix (1, 2, 3) and the first column of the second matrix (1, 10, 100) are highlighted in orange. Curved arrows connect these elements to the top-left element of the result matrix, which is also highlighted in orange and contains the value 321. The other elements in the result matrix are marked with question marks.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 1 & 1 \\ 10 & 2 \\ 100 & 3 \end{bmatrix} = \begin{bmatrix} 321 & ? \\ ? & ? \end{bmatrix}$$

$2 \times 3 \qquad \qquad 3 \times 2 \qquad \qquad 2 \times 2$

Example:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \times \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 10 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix} \times \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 2 \\ 7 \end{bmatrix}$$

Identity matrix

- A special matrix where the input equals the output
- Has 1.0 values in the diagonal and 0.0 everywhere else

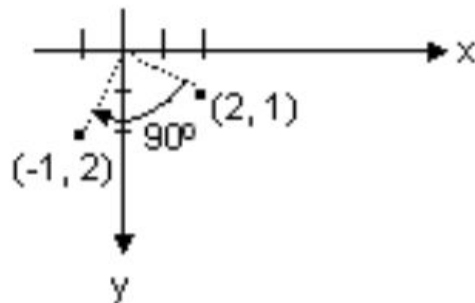
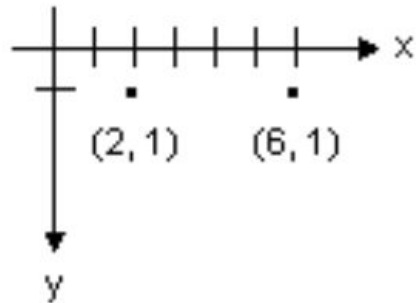
$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Matrix transformations

$$\text{scale}(3, 1) * \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix} * \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 6 \\ 1 \end{pmatrix}$$

$$\text{rotate}(90) * \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} * \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \\ 2 \end{pmatrix}$$



Translations?

We cannot use a 2x2 matrix to translate a vector2.

Translation requires addition, which is **not defined between matrices and vectors**.

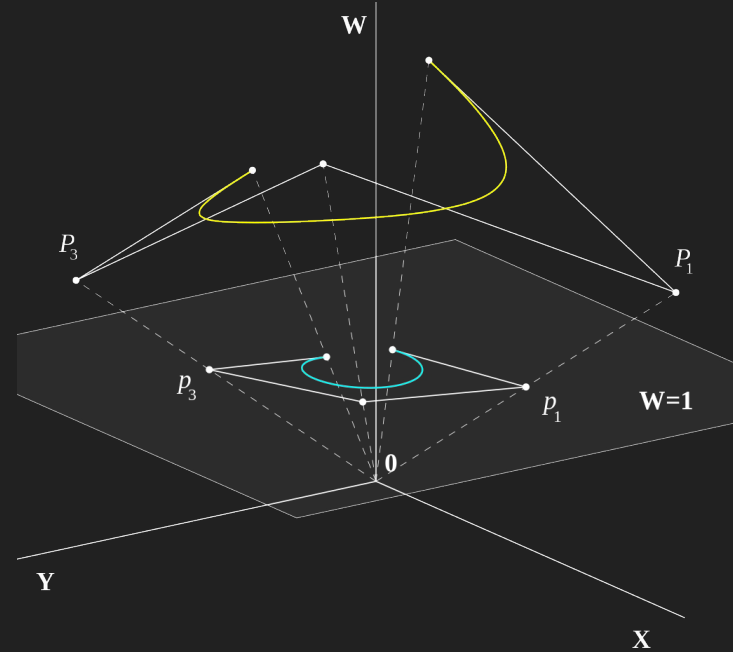
We could just add vectors – but then we lose the single unified matrix transformation.

$$\text{translate}(2, 5) * \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} ? & ? \\ ? & ? \end{pmatrix} * \begin{pmatrix} 3 \\ 4 \end{pmatrix} = * \begin{pmatrix} ? \\ ? \end{pmatrix}$$

$$\text{translate}(2, 5) + \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 2 \\ 5 \end{pmatrix} + \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 5 \\ 9 \end{pmatrix}$$

Homogeneous coordinates

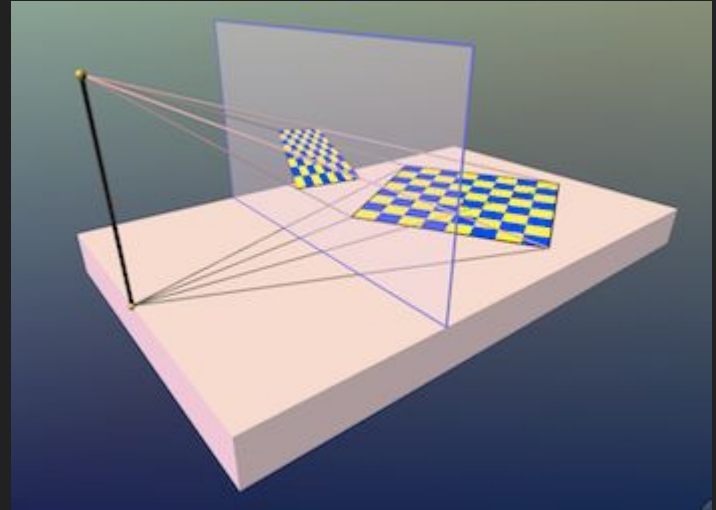
Use **homogeneous coordinates**, where we go up one dimension (e.g. using vec3 to represent vec2)



https://en.wikipedia.org/wiki/Homogeneous_coordinates

Homogeneous coordinates

Use **homogeneous coordinates**, where we go up one dimension (e.g. using vec3 to represent vec2)

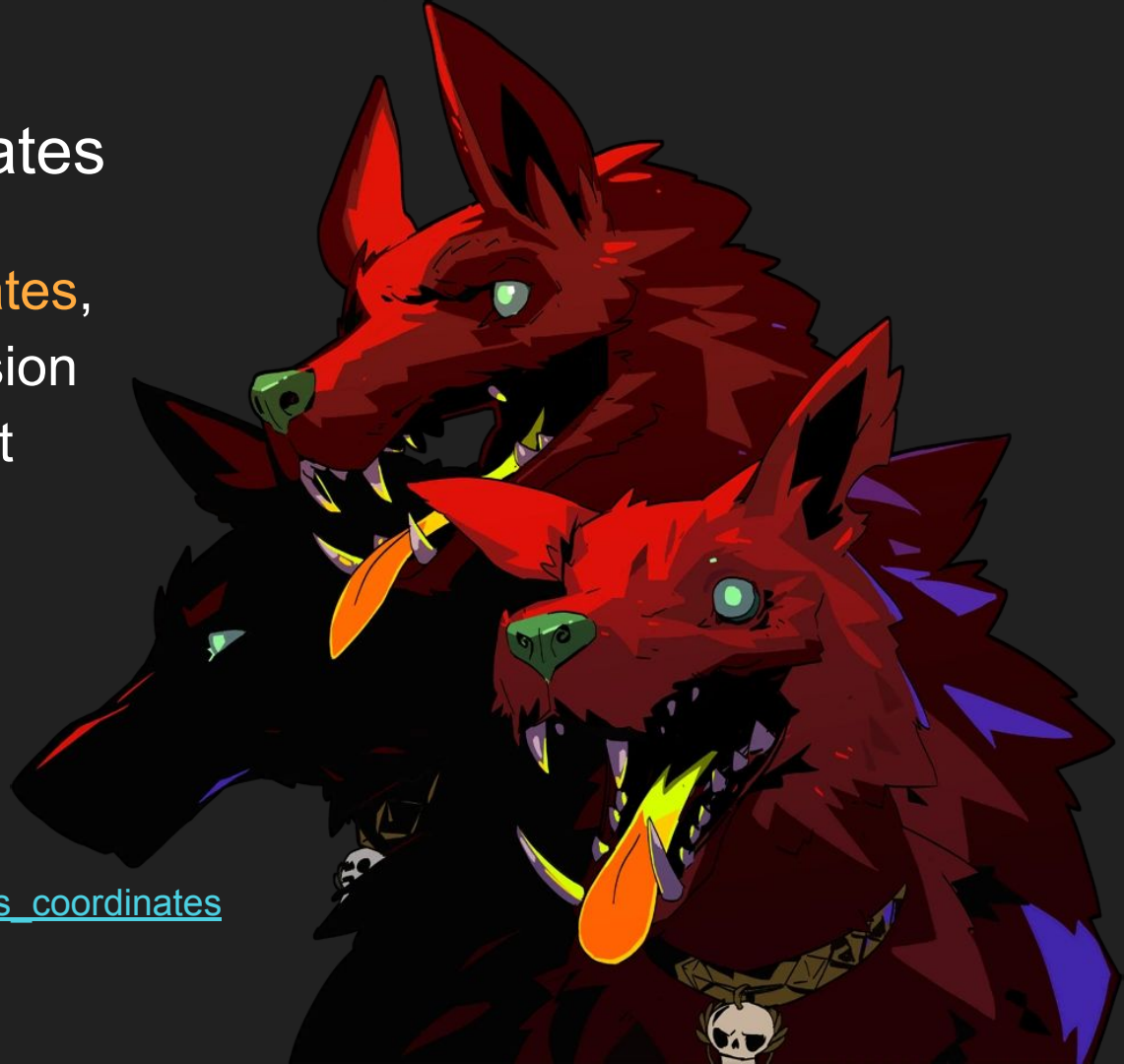


https://en.wikipedia.org/wiki/Homogeneous_coordinates

Homogeneous coordinates

Use **homogeneous coordinates**, where we go up one dimension (e.g. using vec3 to represent vec2)

https://en.wikipedia.org/wiki/Homogeneous_coordinates



Points vs Directions

Where the last element is 1.0 for points and 0.0 for directions.

Point

x=3

y=4

w=1

Direction

x=3

y=4

w=0



Points vs Directions

Where the last element is 1.0 for points and 0.0 for directions.

Point	Direction
-------	-----------

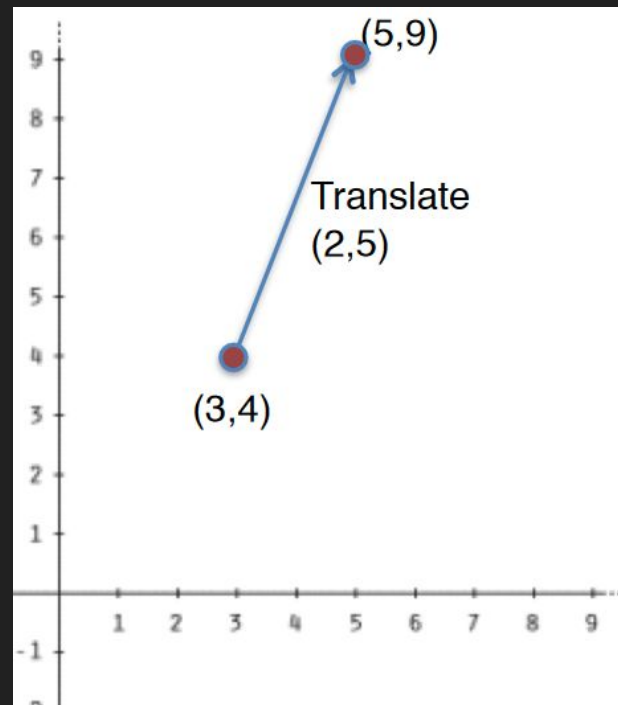
x=3	x=3
-----	-----

y=4	y=4
-----	-----

w=1	w=0
-----	-----

Translations

$$\text{translate}(2,5) * \begin{pmatrix} 3 \\ 4 \\ 1 \end{pmatrix} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 5 \\ 0 & 0 & 1 \end{bmatrix} * \begin{pmatrix} 3 \\ 4 \\ 1 \end{pmatrix} = \begin{pmatrix} 5 \\ 9 \\ 1 \end{pmatrix}$$





GLM usage - matrices

- Matrices are 2D arrays.
- Most common are square matrices of size 2,3,4
- Can also use indexer [][] like nested array.
- Has overloaded operators (+,-,*) for other matrices, vectors and floats
- Constructor with a single 1.0f will create identity matrix.

```
#include <glm/glm.hpp>

int main() {
    glm::vec2 v(5,4);
    glm::mat2 m(1);
    glm::vec2 res = m * v;
}
```

GLM usage - operations



- It is possible to change the numbers of a matrix directly
- Don't !!! There is no need, and you risk to get something nonsensical (also, less performant)
- Instead use some matrix creation functions in glm

```
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>

// we always need to specify the axis
// that we are rotating around.
// in 2D we have only 1:
// the Z axis, "through" the plane
glm::vec3 IN = glm::vec3(0, 0, 1);

int main() {
    glm::mat4 i = glm::mat4(1.0f);

    glm::vec3 mov = glm::vec3(1, 2, 3);
    glm::vec3 scale = glm::vec3(1, 2, 3);
    float rot_angle = 90.0f;

    glm::mat4 t = glm::translate(i, mov);
    glm::mat4 s = glm::scale(i, scale);
    glm::mat4 r = glm::rotate(i, rot_angle, IN);
}
```

GLM usage - transformations



- Transformations are always applied left to right.
- Transformations are not commutative! We'll see examples later.

```
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>

glm::vec3 IN = glm::vec3(0, 0, 1);

int main() {
    // [...]

    // multiplying two matrices "applies" the transf.
    // in the left matrix to the right matrix
    // NOTE: this means that chains of transformations
    //       read from right to left!
    glm::mat4 res = t * r * s * i;

    // // this matrix multiplication returns a
    // // DIFFERENT RESULT!
    // glm::mat4 other = s * r * t * i;
}
```

GLM usage - transformations

Performance tip: you can calculate complex transformations once, cache the result and apply them all to multiple objects very fast.



```
#include <vector>
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>

glm::vec3 IN = vec3(0, 0, 1);

void transform_apply_slow(
    glm::mat4 transforms, int transforms_count
    glm::vec3 mov, float rot, glm::vec3 scale
) {
    for(int i = 0; i < transforms_count; ++i) {
        transforms[i] = glm::scale(transforms[i], scale);
        transforms[i] = glm::rotate(transforms[i], rot, IN);
        transforms[i] = glm::translate(transforms[i], mov);
    }
}

void transform_apply_fast(
    glm::mat4 transforms, int transforms_count
    glm::vec3 mov, float rot, glm::vec3 scale
) {
    // init final transformation to identity matrix
    glm::mat4 t = glm::mat4(1);
    t = glm::scale(t, scale);
    t = glm::rotate(t, rot, IN);
    t = glm::translate(t, mov);
    for(int i = 0; i < transforms_count; ++i)
        transforms[i] = t * transforms[i];
}
```

Agenda

1. Matrices
2. Transforms revised
3. Data-driven Scenes

Transform representations

1. Position, rotation scale (~~today's choice~~)
 - a. Intuitive
 - b. A bit more cumbersome
 - c. A bit less efficient
2. Transform matrices (for 3D exercises)
 - a. They have their specialized math
 - b. Easier to use after understanding them
 - c. More efficient

```
struct Transform
{
    // // not implemented, not worth it
    // Transform* parent;
    vec2f position;
    vec2f scale;
    float rotation;
};

struct Transform3D
{
    Transform3D* parent;

    // pos, rot, scale relative to parent
    // actual data
    glm::mat4 local;

    // pos, rot, scale relative to world
    // computed every frame based on local transform
    // parents' local transforms
    glm::mat4 global;
};
```

Updating the global transform

Our transforms will be stored linearly in memory.
If we have a reference to the parent, this seems pretty easy:

```
for(int i = 0; i < transforms_count; ++i) {  
    glm::mat4 transf = transforms[i].parent->global;  
    transforms[i].global = transf * transforms[i].local;  
}
```

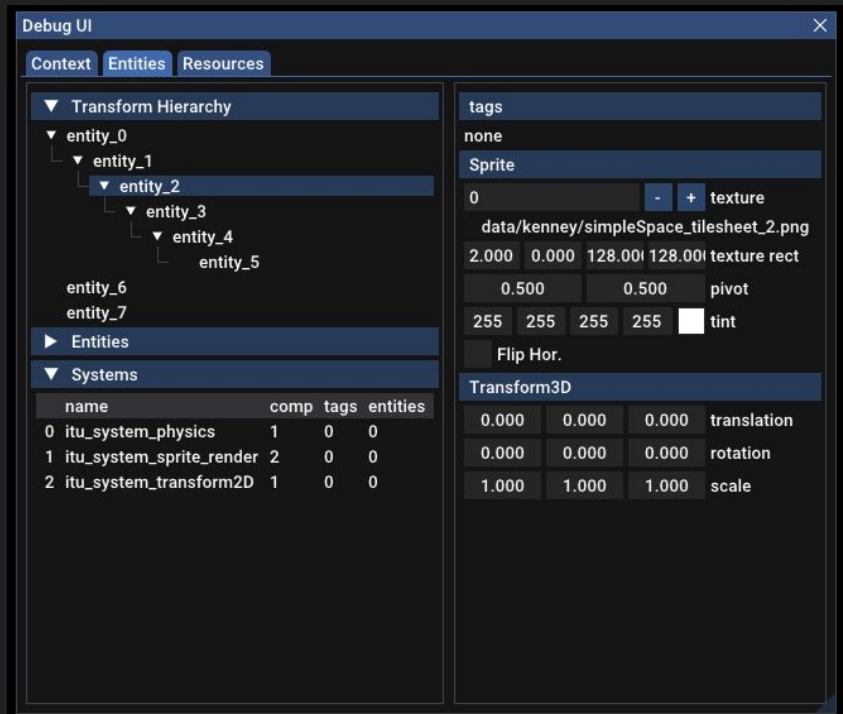
But! This works **ONLY** if the parent's global is already updated!

We can easily achieve this in two ways:

1. sort transforms in breadth order
2. store explicit tree links for breadth-first traversal order

id	parent	local	global
t0	NULL	1, 1, 1	1, 1, 1
t1	t0	0, 0, 0	1, 1, 1
t2	t0	5, 5, 5	6, 6, 6
t3	t1	-1, -1, -1	0, 0, 0

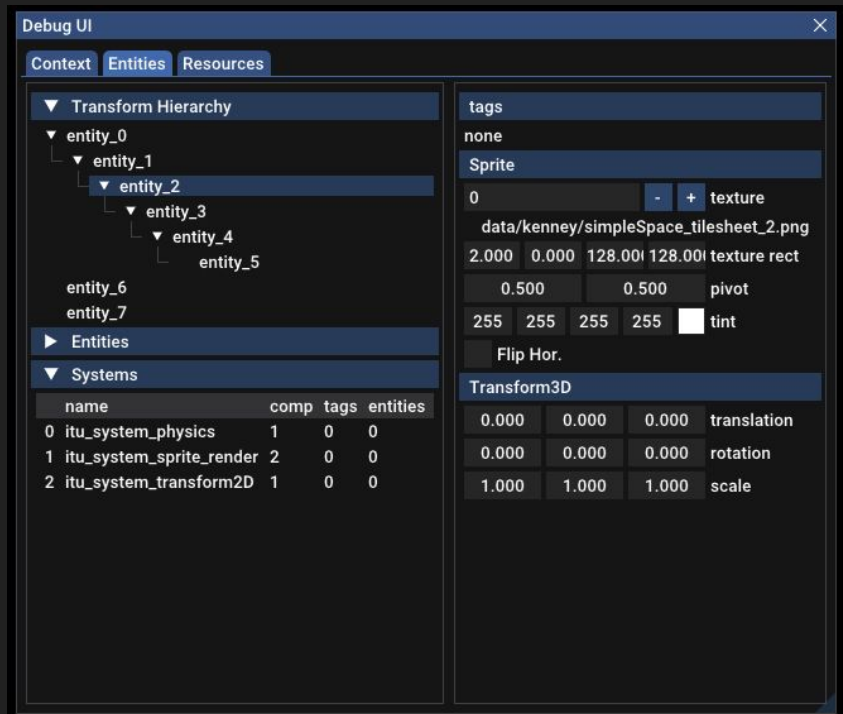
Navigating the transform hierarchy



```
struct Transform3D
{
    // navigational refs for efficient traversal
    // moving a transform in the hierarchy becomes
    // slightly more costly, but that's an operation
    // that should happen relatively rarely
    Transform3D* parent;
    Transform3D* child_first;
    Transform3D* child_last;
    Transform3D* sibling_prev;
    Transform3D* sibling_next;

    glm::mat4 local;
    glm::mat4 global;
};
```

Navigating the transform hierarchy



```
struct Transform3D
{
    // navigational refs for efficient traversal
    // moving a transform in the hierarchy becomes
    // slightly more costly, but that's an operation
    // that should happen relatively rarely
```

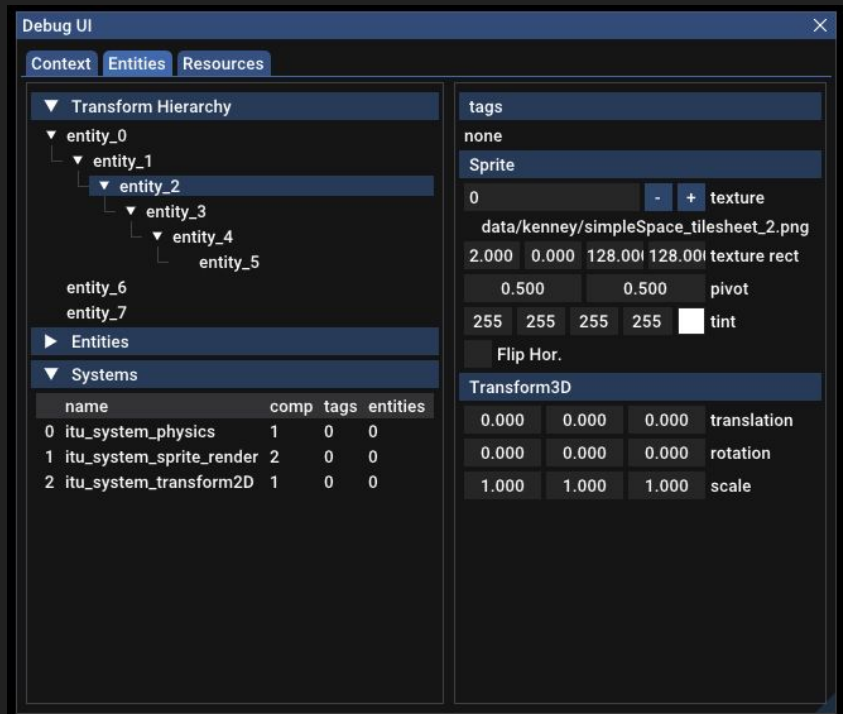
```
    Transform3D* parent;
    Transform3D* child_first;
```

- updating global transform

```
    Transform3D* sibling_next;

    glm::mat4 local;
    glm::mat4 global;
};
```

Navigating the transform hierarchy



```
struct Transform3D
{
    // navigational refs for efficient traversal
    // moving a transform in the hierarchy becomes
    // slightly more costly, but that's an operation
    // that should happen relatively rarely
    Transform3D* parent;
    Transform3D* child_first;
    Transform3D* child_last;
    Transform3D* sibling_prev;
    Transform3D* sibling_next;
};
```

- navigation
 - breadth first
 - depth first (UI rendering)

When do we actually update the global transforms?

Game and engine code will read and write your local transform all the time, but reading the global is more costly! What if one of your parent changed since the last global transform update?

If you make an engine, you will need some sort of wonky way to keep track what your user is doing, and "refresh" the global if it's requested at an inconvenient time.

If you're making your game, you can usually be careful with the timing in which you read/write your transforms, so you can do few bulk updates.

NOTE: updated global transform will be need at least once per frame, for rendering!

https://github.com/godotengine/godot/blob/0fdb93cde6ccb5176f96b0ddbba08d83e6c6aef2/scenes/3d/node_3d.cpp#L641

```
Transform3D Node3D::get_global_transform() const {
    ERR_FAIL_COND_V(!is_inside_tree(), Transform3D());

    /* Due to how threads work at scene level, while this global transform won't be able to be changed from outside a thread,
     * it is possible that multiple threads can access it while it's dirty from previous work. Due to this, we must ensure that
     * the dirty/update process is thread safe by utilizing atomic copies.
     */

    uint32_t dirty = _read_dirty_mask();
    if (dirty & DIRTY_GLOBAL_TRANSFORM) {
        if (dirty & DIRTY_LOCAL_TRANSFORM) {
            _update_local_transform(); // Update local transform atomically.
        }

        Transform3D new_global;
        if (data.parent && !data.top_level) {
            new_global = data.parent->get_global_transform() * data.local_transform;
        } else {
            new_global = data.local_transform;
        }

        if (data.disable_scale) {
            new_global.basis.orthonormalize();
        }

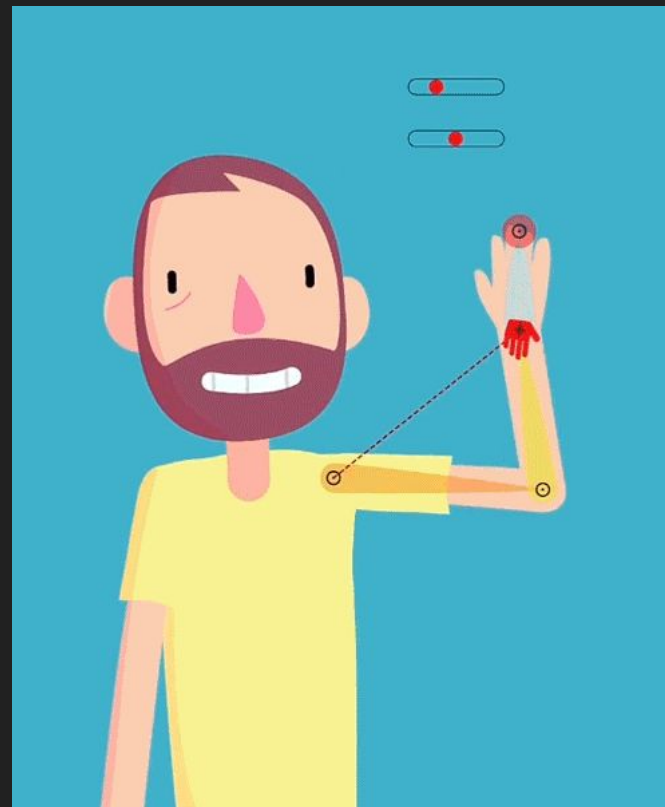
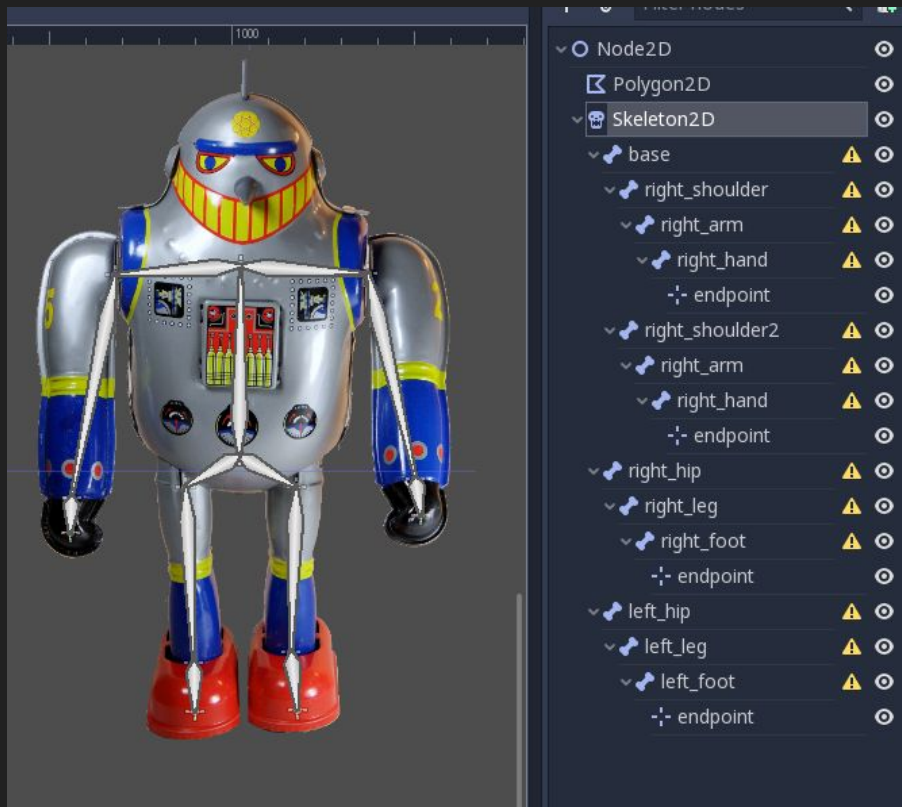
        data.global_transform = new_global;
        _clear_dirty_bits(DIRTY_GLOBAL_TRANSFORM);
    }

    return data.global_transform;
}
```

Agenda

1. Matrices
2. Transforms revised
3. Data-driven Scenes

The GameObject Nesting Problem



Creating a scene using data

```
id_player = itu_entity_create();
itu_entity_set_debug_name(id_player, "player");

Transform transform = TRANSFORM_DEFAULT;
PlayerData data = { 0 };

entity_add_component(id_player, Transform , transform);
entity_add_component(id_player, PlayerData, data);
```

Creating a scene using data

```
id_player = itu_entity_create();  
itu_entity_set_debug_name(id_player, "player");
```

```
Transform transform = TRANSFORM_DEFAULT;  
PlayerData data = { 0 };
```

```
entity_add_component(id_player, Transform , transform);  
entity_add_component(id_player, PlayerData, data);
```



```
{  
  "components": [  
    {  
      "type": 0,  
      "name": "Transform",  
      "data": [{ ... }],  
    },  
    {  
      "type": 17,  
      "name": "PlayerData",  
      "element_size": 48,  
      "data": [  
        {  
          "speed_ground": 5.5,  
          "speed_air": 4.5,  
          "gravity": 7.5,  
        }  
      ],  
    }  
  ]  
}
```

How to represent data

Binary dump

```
Transform transforms[1024];
int transforms_count;

PlayerData players[1024];
int players_count;

// write
SDL_IOStream *fp_w = SDL_IOFromFile("scene.data", "wb+");
SDL_WriteIO(fp_w, transforms, sizeof(Transform) * transforms_count);
SDL_WriteIO(fp_w, players, sizeof(PlayerData) * players_count);

// read
SDL_IOStream *fp_r = SDL_IOFromFile("scene.data", "rb");
SDL_ReadIO(fp_r, players, sizeof(PlayerData) * players_count);
SDL_ReadIO(fp_r, transforms, sizeof(Transform) * transforms_count);
```

Key-based values (example: json)

```
{
  "components": [
    {
      "type": 0,
      "name": "Transform",
      "data": [{ ... }],
    },
    {
      "type": 17,
      "name": "PlayerData",
      "element_size": 48,
      "data": [
        {
          "speed_ground": 5.5,
          "speed_air": 4.5,
          "gravity": 7.5,
        }
      ],
    }
  ]
}
```

Key-based

Pro

- human-readable
- can be built to be resilient to changes between data types between writes and read

Cons

- slow and clunky
- most need full fledged parsing (not all of them)

```
{
  "components": [
    {
      "type": 0,
      "name": "Transform",
      "data": [{ ... }],
    },
    {
      "type": 17,
      "name": "PlayerData",
      "element_size": 48,
      "data": [
        {
          "speed_ground": 5.5,
          "speed_air": 4.5,
          "gravity": 7.5,
        }
      ],
    }
  ]
}
```

Binary dump

Pro

- extremely simple
- extremely fast

Cons

- very annoying to find certain errors in files
- changes to the data structures serialized mean old files are unusable (or require ad-hoc procedures to do conversion)
-

```
Transform transforms[1024];
int transforms_count;

PlayerData players[1024];
int players_count;

SDL_IOStream *fp = SDL_IOFromFile("scene.data", "w+");

// write
SDL_WriteIO(fp, transforms, sizeof(Transform) * transforms_count);
SDL_WriteIO(fp, players, sizeof(PlayerData) * players_count);

// read
SDL_ReadIO(fp_r, players, sizeof(PlayerData) * players_count);
SDL_ReadIO(fp_r, transforms, sizeof(Transform) * transforms_count);
```

How to represent data

Binary dump

```
Transform transforms[1024];  
int transforms_count;
```

```
PlayerData players[1024];  
int players_count;
```

```
// write  
SDL_IOStream *fp_w = SDL_IOFromFile("scene.data", "wb+");  
SDL_WriteIO(fp_w, transforms, sizeof(Transform) * transforms_count);  
SDL_WriteIO(fp_w, players, sizeof(PlayerData) * players_count);  
  
// read  
SDL_IOStream *fp_r = SDL_IOFromFile("scene.data", "rb");  
SDL_ReadIO(fp_r, transforms, sizeof(Transform) * transforms_count);  
SDL_ReadIO(fp_r, players, sizeof(PlayerData) * players_count);
```

Great for shipping games!

- load assets

- savefiles

Key-based values (example: json)

```
{  
  "components": [  
    {  
      "name": "Transform",  
      "data": {  
        "x": 100,  
        "y": 200,  
        "angle": 45,  
        "scale_x": 1.0,  
        "scale_y": 1.0,  
        "rotation": 0  
      }  
    },  
    {  
      "type": "PlayerData",  
      "name": "PlayerData",  
      "element_size": 48,  
      "data": [  
        {  
          "speed_ground": 5.5,  
          "speed_air": 4.5,  
          "gravity": 7.5,  
          "health": 100,  
          "score": 0  
        }  
      ],  
    }  
  ],  
}
```

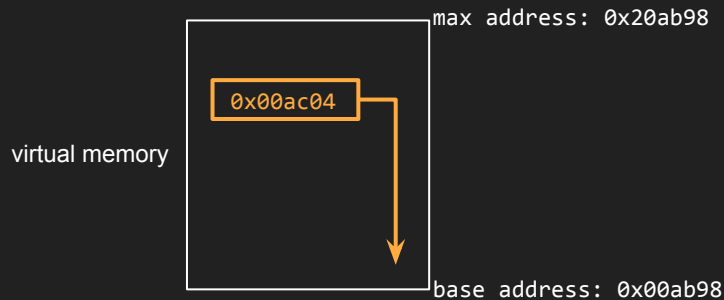
Great for tools!

- save/load assets

- modding

A note on binary headers

1. serialize handles instead of pointers
2. store only what's needed
3. always add headers



A note on binary headers

1. serialize handles instead of pointers
2. store only what's needed
3. always add headers

acceleration structure
(can be recreated from needed data)

needed

if these changes, should be using
key-based system

```
struct ITU_Component
{
    ITU_ComponentType type;
    const char* name;
    Uint64 element_size;
    int count_max;
    int count_alive;
    Uint64* data_loc; // entityID to data loc
    ITU_EntityId* entity_ids; // data loc to entityID
    void* data;
    ITU_ComponendDebugUIRender fn_debug_ui_render;
};
```

A note on binary headers

1. serialize handles instead of pointers
2. store only what's needed
3. always add headers

```
// write
SerializationSceneData serialized_scene; // filled above
SDL_IOStream *fp_write = SDL_IOFromFile("scene.data", "wb+");
SDL_WriteIO(fp_write, &serialized_scene.header, sizeof(SerializationSceneHeader));
SDL_WriteIO(fp_write, serialized_scene.transforms, sizeof(Transform) *
serialized_scene.header.transforms_count);
SDL_WriteIO(fp_write, serialized_scene.players, sizeof(PlayerData)
*serialized_scene.header.players_count);

// read
SerializationSceneData deserialized_scene; // used below
SDL_IOStream *fp_read = SDL_IOFromFile("scene.data", "rb");
SDL_ReadIO(fp_read, &deserialized_scene.header, sizeof(SerializationSceneHeader));
SDL_ReadIO(
    fp_read,
    serialized_scene.transforms,
    sizeof(Transform) * serialized_scene.header.transforms_count
);
SDL_ReadIO(
    fp_read,
    serialized_scene.players,
    sizeof(PlayerData) * serialized_scene.header.players_count
);
```